



Department of Electrical and Computer
Engineering
Instructor : Hassan Tfaily

Lab 2 : “LEDs”

Introduction and objectives

In this week’s lab, you will be introduced to the use of LEDs as OUTPUT peripherals controlled by MC9S12DT256 microcontroller.

This lab focuses on understanding the hardware connections of LEDs on our board, the role of Data Direction Registers (DDR), and how to control LED behavior using CodeWarrior software.

By completing this lab, you will gain hands-on experience in configuring ports, writing programs to control LEDs and demonstrate LED blinking or sequencing using loops and timing delays.

Pre Lab2 :

In this preliminary task, you are asked to answer the following questions :

Ex I:

A **block diagram** consists of blocks that represent different parts of a system connected with signal lines, while a **schematic diagram** shows the components of a system with all the details of interconnection, it is like a blueprint for fabrication.

- Which one of those 2 datasheets , “MC9S12DT256_diagram” or “CSMB12_SCH_A1_0_0” (located in folder ‘Board Diagrams’) represent a block diagram and a schematic diagram ?

- How did the schematic diagram help you in using the board?
- Locate the external oscillator connected to the MCU ,in the schematic diagram, and identify its frequency.
- Deduce the CPU frequency.
- By referring to the datasheet “CPU12RG” (located in folder ‘Instruction’), list the accumulators available in this CPU , and tabulate the differences between them.
(You should compare the size and the function of each accumulator)
- Refer to the “CPU12RG” and determine the range of cycles to execute one instruction [*min cycles : max cycles*].
- Calculate the number of processor cycles of the following program .

```

again          ldx #10
               pshb
               pulb
               pshb
               pulb
               pshb
               pulb
               dex
               bne again
  
```

N.B:

- The “**instruction_set**” datasheet has the complete overview of the instructions for MC9S12DT256 microprocessor.
- Each letter in the **Access Detail column** of the “**CPU12RG**” datasheet represent one CPU cycle as shown in the figure below.

LDAA #oprB	(M) ← A Load Accumulator A	IMM	86 ii	P	P	----	ΔΔ0-
LDAA oprB		DIR	96 dd	rP	rP		
LDAA opr16a		EXT	B6 hh ll	rPO	rOP		
LDAA opm0,xysp		IDX	A6 xb	rP	rP		
LDAA opm9,xysp		IDX1	A6 xb ff	rPO	rPO		
LDAA opm16,xysp		IDX2	A6 xb ee ff	frPP	frPP		
LDAA [D,xysp]		[D]DX	A6 xb	frP	frP		
LDAA [opm16,xysp]		[DX2]	A6 xb ee ff	frPP	frPP		

each letter represent one clock cycle

- h) Knowing the CPU frequency (deduce the period) and the number of CPU cycles of the preceding program, deduce the time needed to run this function.

Ex II:

In the schematic diagram of the board, 4 LEDs are connected to the MCU.

- Follow the LEDs connections and determine the PORT that control them .
- Are the LEDs OUTPUT or INPUT?
- PORTB is a GPIO , its pins can be configured as INPUTs or OUTPUTs . Which register is used to indicate the DATA DIRECTION of those pins?
- Referring to the schematic, are the LEDs connected as active high or active low ?

Lab 2 : “LEDs”

Once you open a new project in CodeWarrior , a **main.asm** file is automatically created in the **Sources** folder. Delete the code between **CLI** and the **Interrupt Vectors** to write your own Code.

Codewarrior

Address of the beginning of the program in Flash EEPROM

```

ROMStart EQU $4000 : absolute address to place any code/constant data

: variable/data section
ORG ROMStart
: Insert here your data definition...
Counter DS.W 1
FiboRes DS.W 1

: code section
ORG ROMStart

Entry:
_Startup: LDS #RAMEnd+1 : initialize the stack pointer
          CLI           : enable interrupts

mainLoop: LDX #1         : X contains counter
counterLoop: STX Counter : update global
            BR   CalcFibo : store result
            LDX Counter
            INX         : larger values cause overflow
            BNE counterLoop : restart
            BRA mainLoop

CalcFibo: : Function to calculate fibonacci numbers. Argument is in X.
          LDX #200      : second last
          LDX #0        : last
          BRB0 X.FiboDone : loop once more (if X was 1, were done already)
FiboLoop: LEAV D,Y       : overwrite second last with new value
          EXG D,Y        : exchange them -> order is correct again
          BRB0 X.FiboLoop
FiboDone: RTS           : result is D

:----- Interrupt Vectors -----
: ORG $FFFF
: DC.W Entry : Reset Vector
  
```

This space is reserved to declare the program variables, which are saved in the RAM starting at @ \$1000.

The program starts after ORG ROMStart, and the first instruction is saved at @ \$4000.

This is an example program, to be deleted in order to write our own program.

Do not modify

Write fully commented programs for the PBMCUSLK board, including appropriate labels, memory operands and constants, to solve the 4 tasks below:

Task1:

The first program should do the following :

- Turn ON the four LEDs.
- Turn ON LED1 only.
- After saving the value of two bytes in memory addresses \$1010 and \$1011, the program will compare them and turn ON LED1 if the content of [\$1010] is greater than the content of [\$1011] , otherwise it will turn ON LED2.

Task2 :

The second program should do the following:

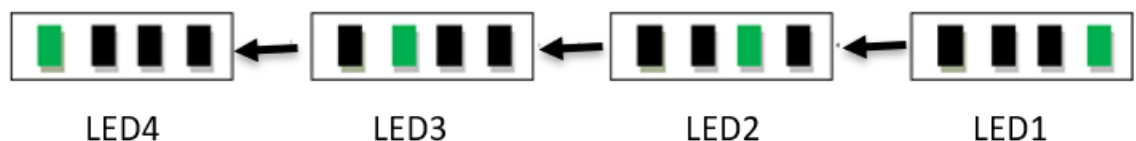
- Turn ON : LED1, LED3 and turn OFF: LED2, LED4 for a delay time of 300ms, then it will turn OFF: LED1, LED3 and turn ON: LED2, LED4.
- Modify the preceding code so that it will run continuously in a loop.

*N.B: to write the **delay** subroutine : calculate the CPU frequency, the number of processor cycles that must be done during the **delay** subroutine to reach the delay time of 300ms .
(use *bsr, dex*).*

Task 3 :

The third program should do the following :

Shift continuously the light to the left from LED1 to LED4 with a delay time of 300ms.(use *lsl* or *rol* instructions).



Task 4:

The final program should do the following:

- a) Define a table of integers in the *'Insert here your data definition'* section of your code.
- b) Search for the maximum number.
- c) Display the result (in binary) on the corresponding LEDs.
